# CPR E 4890: Computer Networking and Data Communications
# Lab Experiment #5 – Error Detection and Go-Back-N ARQ Protocol
# (Total Points: 100)

## Objective

To write code to implement the **sender part** of a Go-Back-N ARQ protocol.

## Pre-Lab

Read the README.md included in provided skeleton project to fully understand the file structure of the project and the header files provided. Then, design appropriate state machines and/or write detailed pseudocode to help implement your solution.

## Tips

- The `recvfrom()` function call is queue based, meaning that if multiple messages have been sent to the corresponding IP and Port it will select the message that arrived first.
- Ensure you have a solid understanding of the Go-Back-N protocol by reviewing lecture slides and notes if necessary. Developing a state machine or pseudo-code before writing code is strongly recommended.

## Lab Expectations

Work through the lab and let the TA know if you have any questions. After the lab, write up a lab report. Please make sure to:

1) **Attend** the lab. **(5 points)**
2) **Summarize** what you learned in a few paragraphs. **(15 points)**
3) **Submit** your well-commented code in a .zip file with your report. **(35 points)**
4) **Demo** your code to the TA. **(30 points)**
5) **Include** your answer to the exercise. **(15 points)**

Submit your **well-commented code** to the TA with your lab report for grading. Submit your report as a PDF file, and your project as a .zip file **alongside** your PDF file. Your PDF file should **NOT** be within the .zip.

## Demonstration Policy

This lab will require you to **demo** your code to the TA. The lab may be demonstrated during:

- **this lab section, or**
- **the TA office hours, or**
- **the regular lab hours on 3/24 (Tue) or 3/25 (Wed)**

## Problem Description

In this lab experiment, you are required to design and develop a sender function (`gbn_sender.c`) to implement a Go-Back-N ARQ protocol on top of UDP. This Go-Back-N ARQ protocol is a **revised version** from the one we learned in class. **One difference is that it uses both ACK and NAK**; this is to simplify the implementation and avoid having to implement the timeout mechanism.

The GBN sender and receiver operate on top of UDP sender and receiver programs (`udp_sender.c`, `udp_receiver.c`). These UDP programs have been completed for you. They create and set up the necessary sockets, then hand control over to their respective GBN subroutines (`gbn_sender.c`, `gbn_receiver.c`). The GBN receiver has also been completed for you, which leaves the `gbn_sender.c` for you to implement.

In addition, two small libraries have been provided for you to work with:

- `pkt_utils.h` contains definitions for a packet struct along with functions for building, printing, and introducing error to your packets
- `crc.h` contains a function which computes a CRC checksum for some data

The initial `gbn_sender.c` contains an example of constructing a packet, introducing error, sending it, and receiving a response. It is up to you to rewrite it to follow GBN behavior and transmit the alphabet, as described in the rest of this document.

**Go-Back-N Sender "gbn_sender.c" (to be completed) (35 points)**

- Send the alphabet, ABCDEFGHIJKLMNOPQRSTUVWXYZ, to the `gbn_receiver` in a total of 13 packets (two characters per packet). NOTE: the total number of packets sent may be higher since the packets may be corrupted by `introduce_bit_error()` and thus need to be retransmitted.

- Use the function `build_packet(...)` found in `pkt_utils.h` to build a packet of the following format:

| Packet Type | Sequence Number | Data | CRC |
|---|---|---|---|
| 1 byte | 1 byte | 2 bytes | 2 bytes |

  - **Packet Type**
    "1": Data Packet (from `gbn_sender` to `gbn_receiver`)
    "2": Acknowledgement Packet (ACK) (from `gbn_receiver` to `gbn_sender`)
    "3": Negative Acknowledgment Packet (NAK) (from `gbn_receiver` to `gbn_sender`)
  - **Sequence Number**
    - Starts from 0 and increments sequentially to 12
  - **Data**
    - Two alphabet characters in each data packet (from `gbn_sender` to `gbn_receiver`)
    - No data is included in ACKs or NAKs, i.e., it shall be set to 0x0000
  - **CRC**
    - CRC generated for this entire packet, including Packet Type, Sequence Number, and Data fields (see the section on CRC)

- After building your sequence of packets, print the uncorrupted packets using `print_packet(...)`

- Apply the `Introduce_bit_error` routine to your packets before transmitting. Pass the BER value to the program as an argument in the command line. NOTE: the `Introduce_bit_error` routine is an in-place function which alters the packet passed in; make sure not to corrupt your source data packets, but only the packets that are being sent out.

- Implement the Go-Back-N ARQ protocol with a send window of size N = 3:
  - The `gbn_sender` maintains a send window of size N = 3
  - The `gbn_sender` transmits any new (unsent) packets in its send window to the receiver. For each sent packet, introduce error to the packet prior to sending, and print the packet along with an indication that it's been sent. The sender must track which packets have been transmitted and are therefore not "new."
  - When the `gbn_sender` receives a cumulative ACK from the `gbn_receiver`, it shall adjust the send window accordingly. It also displays an indication of which packet was positively acknowledged.

- When the `gbn_sender` receives a NAK from the `gbn_receiver`, it retransmits **all** the packets in the send window. This is to emulate the timeout mechanism. It also displays an indication of which packets were negatively acknowledged and were retransmitted.
- The GBN sender will not send any more new packets if it does not have more data to send (e.g. it will not send packets 13 or 14, even if they are within the send window, since it only has data for packets 0-12). The sender may exit once it has received ACK(13), which indicates the receiver has successfully received packets 0-12.

**Go-Back-N Receiver "gbn_receiver.c" (already completed)**

*Note: The `gbn_receiver` function has already been completed and you don't need to make any modifications, but it's a good example to help you build the `gbn_sender` function.*

- Accept data packets from the `gbn_sender`.
- Run the CRC:
  - If the packet is received error-free and in sequence, it displays the packet content and sequence number, then sends back a cumulative ACK.
  - If the packet is received error-free but out of sequence, it does not display the packet content but displays the sequence number, then sends back a cumulative ACK.
  - If the packet is received in error, it does not display the packet content but displays the sequence number, if possible, then sends back a NAK which carries the sequence number that the receiver is waiting for.
  - **ACK and NAK packets are not corrupted nor lost during transmission**.

## Procedure

- Complete the `gbn_sender` function as described above.
- Correctly transmit all 13 packets from the `gbn_sender` to the `gbn_receiver` and observe the sequence of transmissions.
- Correctly handle the transmission errors by implementing proper ACK/NAK processing and data retransmissions.
- Submit your (well-commented) code to the TA with your lab report for grading.

## Exercise (15 points)

Run your program **five** times with each of the following BER values: 0.001, 0.002, 0.005, 0.01, 0.02, 0.05 and plot (Google Sheets or Python works well) the **median** number of transmission attempts per data packet vs. BER value. Summarize your observations.

## Compiling and Running "sender" and "receiver" Programs

A makefile is provided, which allows you to compile and link all programs together with one command:

```
make
```

This creates the sender and receiver binaries in the bin/ directory. Next, run the `receiver`, and have it listen on some port number (the port number does not have to be 4890; it is just an example).

```
bin/receiver 4890
```

Finally, run `sender`, and pass to it the receiver's IP and port number. You must also pass the BER value that will be handed down to your GBN sender.

```
bin/sender 127.0.0.1 4890 0.001
```